2026 联合省选模拟赛

CJZX

时间：2026 年 2 月 25 日 08:30 ～ 13:30

题目名称	2048	狙击	移动石子
题目类型	传统型	传统型	传统型
输入文件名	`merge.in`	`snipe.in`	`stone.in`
输出文件名	`merge.out`	`snipe.out`	`stone.out`
每个测试点时限	1.0 秒	3.0 秒	4.0 秒
内存限制	1024 MiB	1024 MiB	1024 MiB
子任务数目	5	5	3

编译选项

对于 C++ 语言	`-static -O2 -std=c++14`

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. `main` 函数的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
4. 选手提交的程序源文件必须不大于 100KB。
5. 程序可使用的栈空间内存限制与题目的内存限制一致。
6. 全国统一评测时采用的机器配置为：Intel(R) Core(TM) i5-10500 CPU @3.10GHz，内存 8GB。上述时限以此配置为准。
7. 只提供 Linux 格式附加样例文件。
8. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。

2048（merge）

【题目描述】

今天是小 Σ 的生日，她得到了 n 张卡牌作为礼物，每张卡牌上写有一个正整数，第 i 张卡牌上写的数是 2^{a_i}。小 Σ 每次可以选择相邻两张卡牌，满足两张卡牌上写的数 **相同**（设均为 x），并把两张卡牌合并成一张写着 $2x$ 的卡牌，小 Σ 想知道经过若干次（可以是 0 次）操作后，所有卡牌上写的数的最大值最大是多少。

【输入格式】

从文件 ***merge.in*** 中读入数据。
输入的第一行包含一个正整数 n，表示卡牌的个数。
输入的第二行包含 n 个非负整数，其中第 i 个非负整数是 a_i。

【输出格式】

输出到文件 ***merge.out*** 中。
输出一行包含一个非负整数 b，表示所有卡牌上写的数的最大值最大是 2^b。

【样例 1 输入】

```
5
0 1 2 0 1
```

【样例 1 输出】

```
2
```

【样例 1 解释】

小 Σ 不能进行任何操作。

【样例 2 输入】

```
10
7 7 3 2 2 7 7 6 6 7
```

【样例 2 输出】

```
9
```

【样例 2 解释】

小 Σ 可以依次选择第 $(8,9)(8,9)(6,7)(6,7)$ 张卡牌进行操作。

【样例 3】

见到选手目录下的 ***merge3.in*** 与 ***merge3.ans***。
该样例满足子任务 1 的约束。

【样例 4】

见到选手目录下的 ***merge4.in*** 与 ***merge4.ans***。
该样例满足子任务 2 的约束。

【样例 5】

见到选手目录下的 ***merge5.in*** 与 ***merge5.ans***。
该样例满足子任务 3 的约束。

【样例 6】

见到选手目录下的 ***merge6.in*** 与 ***merge6.ans***。
该样例满足子任务 4 的约束。

【样例 7】

见到选手目录下的 ***merge7.in*** 与 ***merge7.ans***。
该样例满足子任务 5 的约束。

【数据范围】

对于所有测试数据，保证：$1 \le n \le 10^5$，$0 \le a_i \le n$。

子任务	$n \le$	特殊性质 A	分值
1	10	否	20
2	500	否	10
3	5000	否	10
4	10^5	是	30
5	10^5	否	30

特殊性质 A：保证 $a_i \le 30$。

狙击（snipe）

【题目描述】

你是一名狙击手，在一个 **三维空间** 内执行狙击任务。

我们用三维坐标 (x, y, z) 表示你的位置，初始 $x = y = z = -10^{100}$。

有 n 个目标，第 i 个目标位于 (a_i, b_i, c_i)。假设你的狙击距离为 K，那么某一时刻位于 (a, b, c) 的目标 **在你的射程内** 当且仅当 $|x - a| \le K, |y - b| \le K, |z - c| \le K$。

在任意时刻，你可以从 (x, y, z) 移动至 $(x + 1, y, z)$ 或 $(x, y + 1, z)$ 或 $(x, y, z + 1)$。你能进行 **任意次** 移动，且可以在 **坐标为整数** 的时刻击杀任意个 **在射程内的目标**。注意，**目标不会移动**。

现在请问，如果要击杀 **所有** n **个目标**，你的狙击距离 K **最小** 可以是多少。

【输入格式】

从文件 ***snipe.in*** 中读入数据。

本题的测试点包含有多组测试数据。

第一行包含一个整数 T，表示测试数据的组数。

接下来依次输入每组测试数据，对于每组测试数据：

- 第一行包含一个整数 n，表示目标的数量。
- 接下来 n 行，每行包含三个整数 a_i, b_i, c_i，表示第 i 个目标的位置。

【输出格式】

输出到文件 ***snipe.out*** 中。

对于每组测试数据，输出一行，包含一个整数，表示答案。

【样例 1 输入】

```
3
2
0 0 0
1 1 1
2
0 1 0
1 0 1
5
1 1 4
5 1 4
```

```
1 9 1
9 8 1
0 0 0
```

【样例 1 输出】

```
0
1
2
```

【样例 2】

见选手目录下的 ***snipe2.in*** 与 ***snipe2.ans***。
该样例满足子任务 1 的约束。

【样例 3】

见选手目录下的 ***snipe3.in*** 与 ***snipe3.ans***。
该样例满足子任务 2 的约束。

【样例 4】

见选手目录下的 ***snipe4.in*** 与 ***snipe4.ans***。
该样例满足子任务 3 的约束。

【样例 5】

见选手目录下的 ***snipe5.in*** 与 ***snipe5.ans***。
该样例满足子任务 4 的约束。

【样例 6】

见选手目录下的 ***snipe6.in*** 与 ***snipe6.ans***。
该样例满足子任务 5 的约束。

【样例 7】

见选手目录下的 ***snipe7.in*** 与 ***snipe7.ans***。
该样例满足子任务 5 的约束。

【数据范围】

对于所有测试数据，保证：$1 \le T \le 5 \times 10^4$，$1 \le n \le 5 \times 10^5$，$-10^9 \le a_i, b_i, c_i \le 10^9$，$\sum n \le 2 \times 10^6$。

子任务	$n \le$	$\sum n \le$	特殊性质 A	特殊性质 B	分值
1	250	500	否	否	15
2	1500	3000	否	否	15
3	5×10^5	2×10^6	是	否	15
4	5×10^5	2×10^6	否	是	15
5	5×10^5	2×10^6	否	否	40

特殊性质 A：保证 $-300 \le a_i, b_i, c_i \le 300$。

特殊性质 B：保证 $c_i = 0$。

移动石子（stone）

【题目描述】

现有一张包含 n 个顶点和 m 条边的无向图。

初始时，某些顶点上有石子，你需要将这些石子移动到一个目标状态。注意，**石子不可区分**。

你可以执行以下操作 **任意次**：

- 对于每个位于 u_i 的石子，选择一个相邻顶点 v_i。**同时** 将所有石子从各自的 u_i 移动到对应的 v_i。

我们称一个操作序列合法，当其仅当在任何时刻，每个顶点最多只包含一个石子。请判断是否存在合法的操作序列可以将石子**从初始状态移动到目标状态**。若存在，请构造一个合法的操作序列。

我们会根据你对是否有解的判断，与你构造的操作序列是否合法、是否能将石子从初始状态移动到目标状态以及操作序列的长度进行评分，详见评分方式部分。

【输入格式】

从文件 ***stone.in*** 中读入数据。

本题的测试点包含有多组测试数据。

第一行包含一个整数 T，表示测试数据的组数。

接下来依次输入每组测试数据，对于每组测试数据：

- 第一行包含两个整数 n 和 m，表示图中顶点数和边数。
- 第二行包含一个长为 n 的由 0 和 1 组成的二进制字符串 s。s 的第 i 位表示初始状态下第 i 个顶点上的石子数量。
- 第三行包含一个长为 n 的由 0 和 1 组成的二进制字符串 t。t 的第 i 位表示目标状态下第 i 个顶点上的石子数量。
- 接下来的 m 行每行包含两个整数 u 和 v，表示第 u 个顶点与第 v 个顶点之间有一条无向边。

【输出格式】

输出到文件 ***stone.out*** 中。

对于每个测试数据：

- 在第一行输出 `Yes` 或 `No` 表示是否有解。你可以任意使用大写或小写输出答案。比如，输出 `yEs`、`yes`、`Yes`、`YES` 均视为输出有解。
- 如果存在合法的操作序列，你需要输出你构造的操作序列。
  - 在第二行输出一个整数 k 表示操作次数。

- 假设初始状态包含 c 个石子，接下来的 $k+1$ 行每行输出 c 个整数，表示初始状态和每次操作后石子位置的集合，满足：
  * 每行输出的 c 个整数互不相同。
  * 第一行输出的石子位置与输入中的初始状态匹配，顺序可任意。
  * 对于所有 i（$1 \le i \le k$）和 j（$1 \le j \le c$），满足第 i 行第 j 个整数与第 $i+1$ 行第 j 个整数对应的顶点在图中是相邻的。

【样例 1 输入】

```
4
2 1
10
01
1 2
11 11
11011001010
01101011100
1 2
2 3
3 4
4 5
5 6
6 7
7 8
8 9
9 10
10 11
11 1
3 2
110
101
1 2
2 3
3 2
111
111
```

```
1 2
2 3
```

样例 **1** 输出

```
Yes
1
1
2
Yes
6
1 2 4 5 8 10
2 3 5 6 9 11
3 2 6 7 10 1
4 3 7 8 11 2
5 2 8 9 1 3
6 3 7 8 2 4
7 2 8 9 3 5
No
Yes
0
1 2 3
```

【样例 2】

见选手目录下的 ***stone2.in*** 与 ***stone2.ans***。
该样例满足子任务 3 的约束。

【样例 3】

见选手目录下的 ***stone3.in*** 与 ***stone3.ans***。
该样例满足 $n \le 20$。

【数据范围】

对于所有测试数据，保证：

- $T \le 1000$，$n \le 2000$，$0 \le m \le \min(\frac{n(n-1)}{2}, 10^4)$，$\sum n \le 2000$，$\sum m \le 10^4$，$1 \le u, v, \le n$；
- 保证图是简单图（没有自环和重边）；

- 保证 s 和 t 中 1 的数量相同。

子任务	$\sum n \le$	特殊性质 A	分值
1	30	无	20
2	100	无	20
3	2000	有	30
4	2000	无	30

特殊性质 A：保证图是一个森林。

【评分方式】

对于每个测试点，评分方式如下：

- 如果在任意一组测试数据中，你错误判断了是否有解，或在有解的情况下，你输出的操作序列不合法，则获得 0 分。注意，即使你只希望判断是否有解，也需要在有解的情况下输出一个合法的操作序列。一个简单的方式是在第二行输出 0，并在第三行输出初始状态。
- 如果在所有测试数据中，你输出的操作序列满足 $\sum n(k+1) > 10^7$，则获得 0 分。
- 在上述条件基础上：
  - 你将立即获得 30% 的分数。
  - 如果在每一组有解的测试数据中，你输出的方案的最后一行与目标状态都匹配：
    - 如果在每一组有解的测试数据中，你构造的操作序列均满足 $k \le 2n^2$，则可以再获得另外 40% 的分数。
    - 如果在每一组有解的测试数据中，你构造的操作序列均满足 $k \le 2n$，则可以再获得另外 30% 的分数。

可以证明，如果存在合法的操作序列，则必然存在操作次数不超过 $2n$ 的合法操作序列。

对于每个子任务，得分是这个子任务中每个测试点的得分的最小值。